

Optimization

2.6 Numerical approximation of derivatives

José Firmino Aguilar Madeira

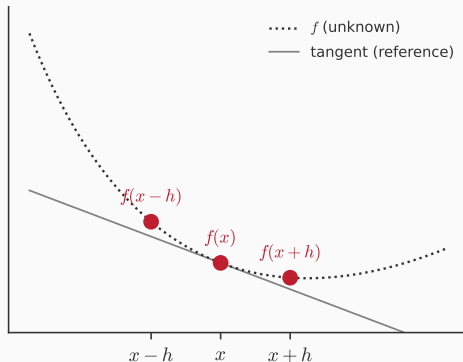
What if we can only evaluate f ?

We have a **simulator**: we give it x and it returns $f(x)$. Nothing else — neither $f'(x)$ nor $f''(x)$.

Newton (2.5) wants f' and f'' ; bisection (2.4) wants f' . But we can ask the simulator for

$$f(x - h), \quad f(x), \quad f(x + h).$$

How much can we find out from these three numbers?



Guiding example $f(x) = x^2 + 54/x$, $x = 2.5$; the tangent only as a reference.

What if we can only evaluate f ?

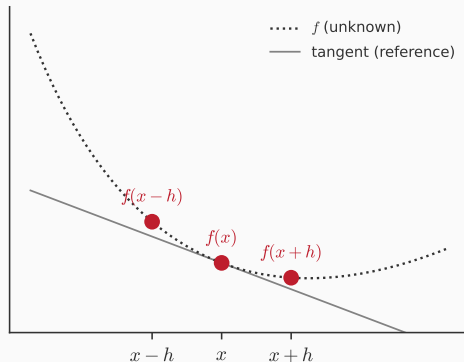
We have a **simulator**: we give it x and it returns $f(x)$. Nothing else — neither $f'(x)$ nor $f''(x)$.

Newton (2.5) wants f' and f'' ; bisection (2.4) wants f' . But we can ask the simulator for

$$f(x - h), \quad f(x), \quad f(x + h).$$

How much can we find out from these three numbers?

More than it seems: the slope and the curvature of f at x , approximately. Approximating derivatives from nearby values of f : these are **finite differences**.



Guiding example $f(x) = x^2 + 54/x$, $x = 2.5$; the tangent only as a reference.

First idea: measure the slope

With two points, $(x, f(x))$ and $(x + h, f(x + h))$, the slope of the **secant** is

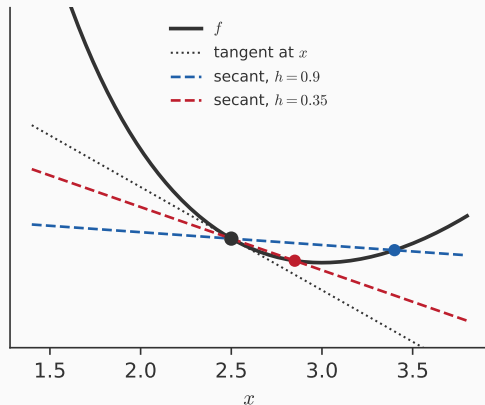
$$\frac{f(x + h) - f(x)}{h}.$$

As $h \rightarrow 0$, the secant approaches the tangent.

Hence

$$f'(x) \approx \frac{f(x + h) - f(x)}{h}$$

— the *forward difference*.



First idea: measure the slope

With two points, $(x, f(x))$ and $(x + h, f(x + h))$, the slope of the **secant** is

$$\frac{f(x + h) - f(x)}{h}.$$

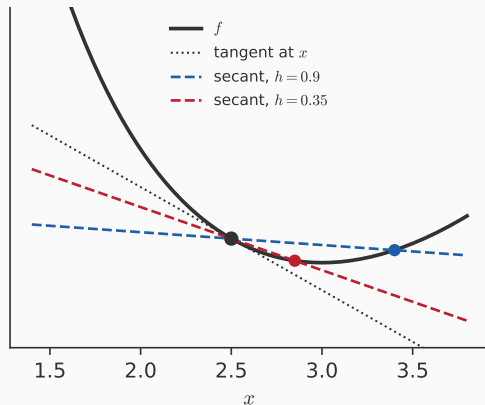
As $h \rightarrow 0$, the secant approaches the tangent.
Hence

$$f'(x) \approx \frac{f(x + h) - f(x)}{h}$$

— the *forward difference*.

But we only look at **one side** of x .

What if we use information from both sides?



Why is the central difference better?

Taylor around x :

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(x) + O(h^4).$$

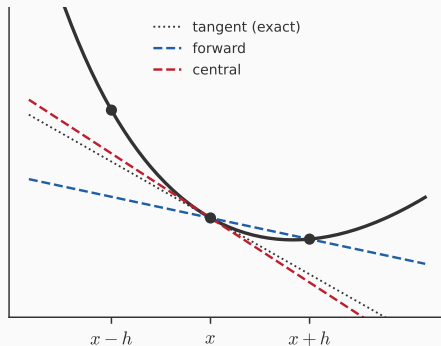
Subtracting the expansions, even terms cancel:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad \text{error } O(h^2).$$

The forward difference leaves the term $\frac{h}{2}f''(x)$: error $O(h)$.

In the guiding example, with the *same* $h = 0.025$:
forward -3.5295 (error 0.11); central -3.64086
(error $9 \cdot 10^{-4}$).

Backward, $[f(x) - f(x-h)]/h$: also $O(h)$ (useful if only one side can be evaluated).



And the second derivative?

The same Taylor expansion, but now **adding** (the odd terms cancel):

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + O(h^4)$$
$$\Rightarrow \boxed{f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}} \quad \text{error } O(h^2).$$

$$\boxed{f(x-h), f(x), f(x+h)} \longrightarrow \boxed{f'(x) \text{ and } f''(x)}$$

The **same three values** give both derivatives. Therefore

numerical Newton in 1D: **3 evaluations of f** per point

— the count that 2.5 had already anticipated. Bisection would need only f' : 2 evaluations ($f(x \pm h)$).

A calculation by hand

$f(x) = x^2 + 54/x$, $x = 2.5$, $h = 0.025$ ($= 0.01|x|$). Exact values: $f'(2.5) = -3.64$, $f''(2.5) = 8.912$.

$f(2.475)$	$f(2.5)$	$f(2.525)$
27.94380682	27.85	27.76176361

$$f'_{\text{num}} = \frac{27.76176361 - 27.94380682}{0.05} = -3.64086,$$

$$f''_{\text{num}} = \frac{27.76176361 - 2 \cdot 27.85 + 27.94380682}{0.000625} = 8.9127.$$

3 evaluations of $f \Rightarrow$ two derivatives with error $< 10^{-3}$

And with these values, the Newton step of 2.5: $x_1 = 2.5 + 3.64086/8.9127 = 2.9085$ (exact: 2.9084).

8 decimal places are needed: with 6 (27.943807, 27.761764), f''_{num} would give 8.9136 — this is cancellation (see next).

So is it enough to make h ever smaller?

It seems so: the error is $O(h^2)$. In double precision, in the same example:

h	central f'	error	central f''	error
0.1	-3.65385	$1.4 \cdot 10^{-2}$	8.9231	$1 \cdot 10^{-2}$
0.025	-3.64086	$9 \cdot 10^{-4}$	8.9127	$7 \cdot 10^{-4}$
10^{-3}	-3.64000	$1.4 \cdot 10^{-6}$	8.9120	$1 \cdot 10^{-6}$

So is it enough to make h ever smaller?

It seems so: the error is $O(h^2)$. In double precision, in the same example:

h	central f'	error	central f''	error
0.1	-3.65385	$1.4 \cdot 10^{-2}$	8.9231	$1 \cdot 10^{-2}$
0.025	-3.64086	$9 \cdot 10^{-4}$	8.9127	$7 \cdot 10^{-4}$
10^{-3}	-3.64000	$1.4 \cdot 10^{-6}$	8.9120	$1 \cdot 10^{-6}$
10^{-6}	-3.64000	$1 \cdot 10^{-9}$	8.9067	$5 \cdot 10^{-3}$
10^{-8}	-3.64000	$5 \cdot 10^{-8}$	0.0000	9

With $h = 10^{-8}$ the computed second derivative is **zero** — the true one is 8.912. And the error of f' grows again.

No: too small an h is also bad.

Two conflicting errors

Truncation (from the formula): $E_T \sim K h^2$. We want h **small**.

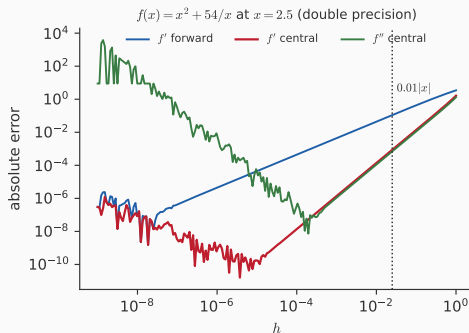
Round-off: $f(x+h)$ and $f(x-h)$ are almost equal; each has an error $\approx \varepsilon_m |f|$, and the difference is divided by h (or h^2):

$$E_R \sim \frac{\varepsilon_m |f|}{h} \quad (f'), \quad E_R \sim \frac{\varepsilon_m |f|}{h^2} \quad (f'').$$

We want h **large**. There is an optimal intermediate h :

$$h_{\text{opt}}^{f'} \sim \varepsilon_m^{1/3} \times \text{scale}, \quad h_{\text{opt}}^{f''} \sim \varepsilon_m^{1/4} \times \text{scale}$$

In double precision ($\varepsilon_m \approx 2.2 \cdot 10^{-16}$): $\varepsilon_m^{1/3} \approx 6 \cdot 10^{-6}$, $\varepsilon_m^{1/4} \approx 1.2 \cdot 10^{-4}$ — *orders of magnitude*; the scale depends on $|f|$, on the derivatives and on x , not only on $|x|$.



Which h do we use, then?

In the course exercises

$$h = \begin{cases} 0.01 |x|, & |x| > 0.01, \\ 10^{-4}, & \text{otherwise.} \end{cases}$$

h **relative** to the scale of x , with an absolute minimum near zero.

A deliberately **conservative** step: large enough to avoid round-off and suitable for *guiding* an iterative method. The truncation error in f' is $\approx h^2 |f'''|/6$: near x^* , where $f' \rightarrow 0$, it is no longer small compared with f' (next page).

For high accuracy

Choose h taking into account the machine precision, the scale of f and of x , and the order of the formula; the values on the previous page give the order of magnitude. Or reduce h in the last iterations.

1 % is **not** a universally optimal step:
it is a working rule.

Newton with numerical derivatives: $f(x) = x^2 + 54/x$, $x_0 = 1$

Rule $h = 0.01|x|$; stopping $|f'_{\text{num}}| < \varepsilon_g = 10^{-4}$.

k	x_k	h	f'_{num}	f''_{num}	x_{k+1}
0	1.00000	0.0100	-52.0054	110.0108	1.47273

- Each row costs $n_f = 3$: $f(x)$, $f(x \pm h)$. It stops at $k = 6$ and returns x_7 : 7 rows, $n_f = 21$, +1 ($f(x_7)$), $n_f = 22$.
- The first iterations agree with exact Newton to 3 digits; the difference is at the end.
- Reducing h to $10^{-4}|x|$ at the end, one iteration gives 3.00000000: h can be **adaptive**.

Newton with numerical derivatives: $f(x) = x^2 + 54/x$, $x_0 = 1$

Rule $h = 0.01|x|$; stopping $|f'_{\text{num}}| < \varepsilon_g = 10^{-4}$.

k	x_k	h	f'_{num}	f''_{num}	x_{k+1}
0	1.00000	0.0100	-52.0054	110.0108	1.47273
1	1.47273	0.0147	-21.9541	35.8141	2.08573

- Each row costs $n_f = 3$: $f(x)$, $f(x \pm h)$. It stops at $k = 6$ and returns x_7 : 7 rows, $n_f = 21$, +1 ($f(x_7)$), $n_f = 22$.
- The first iterations agree with exact Newton to 3 digits; the difference is at the end.
- Reducing h to $10^{-4}|x|$ at the end, one iteration gives 3.0000000: h can be **adaptive**.

Newton with numerical derivatives: $f(x) = x^2 + 54/x$, $x_0 = 1$

Rule $h = 0.01|x|$; stopping $|f'_{\text{num}}| < \varepsilon_g = 10^{-4}$.

k	x_k	h	f'_{num}	f''_{num}	x_{k+1}
0	1.00000	0.0100	-52.0054	110.0108	1.47273
1	1.47273	0.0147	-21.9541	35.8141	2.08573
2	2.08573	0.0209	-8.2428	13.9040	2.67857

- Each row costs $n_f = 3$: $f(x)$, $f(x \pm h)$. It stops at $k = 6$ and returns x_7 : 7 rows, $n_f = 21$, +1 ($f(x_7)$), $n_f = 22$.
- The first iterations agree with exact Newton to 3 digits; the difference is at the end.
- Reducing h to $10^{-4}|x|$ at the end, one iteration gives 3.0000000: h can be **adaptive**.

Newton with numerical derivatives: $f(x) = x^2 + 54/x$, $x_0 = 1$

Rule $h = 0.01|x|$; stopping $|f'_{\text{num}}| < \varepsilon_g = 10^{-4}$.

k	x_k	h	f'_{num}	f''_{num}	x_{k+1}
0	1.00000	0.0100	-52.0054	110.0108	1.47273
1	1.47273	0.0147	-21.9541	35.8141	2.08573
2	2.08573	0.0209	-8.2428	13.9040	2.67857
3	2.67857	0.0268	-2.1700	7.6203	2.96334

- Each row costs $n_f = 3$: $f(x)$, $f(x \pm h)$. It stops at $k = 6$ and returns x_7 : 7 rows, $n_f = 21$, +1 ($f(x_7)$), $n_f = 22$.
- The first iterations agree with exact Newton to 3 digits; the difference is at the end.
- Reducing h to $10^{-4}|x|$ at the end, one iteration gives 3.00000000: h can be **adaptive**.

Newton with numerical derivatives: $f(x) = x^2 + 54/x$, $x_0 = 1$

Rule $h = 0.01|x|$; stopping $|f'_{\text{num}}| < \varepsilon_g = 10^{-4}$.

k	x_k	h	f'_{num}	f''_{num}	x_{k+1}
0	1.00000	0.0100	-52.0054	110.0108	1.47273
1	1.47273	0.0147	-21.9541	35.8141	2.08573
2	2.08573	0.0209	-8.2428	13.9040	2.67857
3	2.67857	0.0268	-2.1700	7.6203	2.96334
4	2.96334	0.0296	-0.2233	6.1507	2.99965

- Each row costs $n_f = 3$: $f(x)$, $f(x \pm h)$. It stops at $k = 6$ and returns x_7 : 7 rows, $n_f = 21$, +1 ($f(x_7)$), $n_f = 22$.
- The first iterations agree with exact Newton to 3 digits; the difference is at the end.
- Reducing h to $10^{-4}|x|$ at the end, one iteration gives 3.00000000: h can be **adaptive**.

Newton with numerical derivatives: $f(x) = x^2 + 54/x$, $x_0 = 1$

Rule $h = 0.01|x|$; stopping $|f'_{\text{num}}| < \varepsilon_g = 10^{-4}$.

k	x_k	h	f'_{num}	f''_{num}	x_{k+1}
0	1.00000	0.0100	-52.0054	110.0108	1.47273
1	1.47273	0.0147	-21.9541	35.8141	2.08573
2	2.08573	0.0209	-8.2428	13.9040	2.67857
3	2.67857	0.0268	-2.1700	7.6203	2.96334
4	2.96334	0.0296	-0.2233	6.1507	2.99965
5	2.99965	0.0300	-0.0027	6.0018	3.00010

- Each row costs $n_f = 3$: $f(x)$, $f(x \pm h)$. It stops at $k = 6$ and returns x_7 : 7 rows, $n_f = 21$, +1 ($f(x_7)$), $n_f = 22$.
- The first iterations agree with exact Newton to 3 digits; the difference is at the end.
- Reducing h to $10^{-4}|x|$ at the end, one iteration gives 3.00000000: h can be **adaptive**.

Newton with numerical derivatives: $f(x) = x^2 + 54/x$, $x_0 = 1$

Rule $h = 0.01|x|$; stopping $|f'_{\text{num}}| < \varepsilon_g = 10^{-4}$.

k	x_k	h	f'_{num}	f''_{num}	x_{k+1}
0	1.00000	0.0100	-52.0054	110.0108	1.47273
1	1.47273	0.0147	-21.9541	35.8141	2.08573
2	2.08573	0.0209	-8.2428	13.9040	2.67857
3	2.67857	0.0268	-2.1700	7.6203	2.96334
4	2.96334	0.0296	-0.2233	6.1507	2.99965
5	2.99965	0.0300	-0.0027	6.0018	3.00010
6	3.00010	0.0300	$-4 \cdot 10^{-7}$	6.0000	3.00010

It stagnates at 3.0001: with $h = 0.03$, $f'_{\text{num}}(3) \approx -6 \cdot 10^{-4} \neq 0$.

Exact Newton is quadratically convergent; with approximate derivatives of fixed accuracy, the acceleration stops when the error of the derivatives becomes dominant.

the method is only as accurate as the information we give it

- Each row costs $n_f = 3$: $f(x)$, $f(x \pm h)$. It stops at $k = 6$ and returns x_7 : 7 rows, $n_f = 21$, +1 ($f(x_7)$), $n_f = 22$.
- The first iterations agree with exact Newton to 3 digits; the difference is at the end.
- Reducing h to $10^{-4}|x|$ at the end, one iteration gives 3.0000000: h can be **adaptive**.

The core, in MATLAB:

```
h = max(0.01*abs(x), 1e-4);  
fm = f(x - h); f0 = f(x); fp = f(x + h);  
df = (fp - fm) / (2*h);  
ddf = (fp - 2*f0 + fm) / h^2;  
x = x - df/ddf;           % Newton step
```

The full functions (NumDerivs, NewtonNumeric) are in the course companion code.

3 evaluations $\Rightarrow f'_{\text{num}}$ and f''_{num}

To choose the half, bisection needs only the *sign* of f' , that of $f(x+h) - f(x-h)$ (2 evaluations); for the test $|f'(z)| \leq \varepsilon$, divide by $2h$. Near the zero, that sign may be affected by numerical noise (2.4).

Summary table — Chapter 2

What information do we have? How much does it cost to obtain? What can we guarantee with it?

Method (idea)	Information	Cost per iteration	Guarantee	Main limitation
2.2 Golden section: eliminate interval	f	$n_f = 1$ (+2 initial)	linear (0.618), robust (f unimodal)	slow: ≈ 4.8 reductions per digit
2.3 Powell: interpolate a parabola	f	$n_f = 1$ (+3 initial; +1 per reflection)	local superlinear; on its own, none	needs safeguards ($a_2 \leq 0$)
2.4 Bisection: locate the zero by its sign	f'	$n_g = 1$ ($+f'(a), f'(b)$)	linear (0.5), robust with a single bracketed zero	needs $f'(a) < 0 < f'(b)$; uses only the sign
2.5 Newton: estimate the zero (linearizes f')	f', f''	$n_g = 1, n_H = 1$	local quadratic, under regularity ($f''(x^*) \neq 0$)	may diverge or go to a maximum
2.6 Finite diff.: approximate f', f''	f	$n_f = 3$ per point (numerical Newton)	truncation error $O(h^2)$	round-off if h is small

KEY TAKEAWAYS — 2.6

1. Nearby values of f approximate derivatives; the central formulas have error $O(h^2)$:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \quad f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}.$$

2. The same three values $f(x-h)$, $f(x)$, $f(x+h)$ give f' and f'' : numerical Newton in 1D costs **3 evaluations of f** per point.
3. Large $h \Rightarrow$ truncation error; small $h \Rightarrow$ round-off (cancellation). There is a trade-off.
4. **An algorithm cannot be more accurate than the information it receives.**

KEY TAKEAWAYS — Chapter 2

1. **Shrink a region that contains the solution** (golden section, bisection): predictable, slow; robust (guaranteed bracketing).
2. **Estimate directly where the solution is** (successive quadratic interpolation, Newton): very fast nearby, no guarantee far away.
3. In practice they are combined: `fminbnd` (golden section + parabola); bisection + Newton.
4. The usual questions: what information do we have? How much does it cost to obtain? What can we guarantee with it?
5. In 1D we decide **how far to go**.

Next (Ch. 3): in several variables, first **where** to go.

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Deb, K., *Optimization for Engineering Design: Algorithms and Examples*, 2nd ed., PHI Learning, 2012 — Ch. 2.
- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2nd ed., Springer, 2006 — Ch. 8, numerical derivatives.

Resources

- Companion code: NumDerivs, NewtonNumeric. MATLAB: fminbnd.